

SS-BDD: Automated Acceptance Testing for Spreadsheets

Lara Almeida
Federal University of
São João del-Rei

lara.araujomendes20@gmail.com

Elder Cirilo
Federal University of
São João del-Rei
elder@ufs.edu.br

Eiji Adachi Barbosa
Federal University of
Rio Grande do Norte
eijiadachi@imd.ufrn.br

ABSTRACT

Current Spreadsheet Applications, such as Excel and Google Sheets, provide innumerable built-in facilities, including arithmetic, financial and statistical operations, as well as conditional expressions. Thus, users with little or no formal training in programming can use Spreadsheet Applications to implement their own Spreadsheet Programs. In fact, Spreadsheet Applications have become one of the most popular end-user programming environments nowadays. However, these applications also ease the introduction of errors in Spreadsheet Programs. Minor mistakes in formulas can mislead decision-making processes, resulting in uncountable costs to organizations. In general, end-user programmers are unaware of the potential risks that the uncontrolled construction of Spreadsheet Programs can cause. Therefore, a major focus of this paper is to offer an automated approach that makes programmers aware of introduced faults, so that they can build high quality Spreadsheet Programs. In particular, we propose SS-BDD, a framework for building and running Spreadsheets test scenarios, which relies on the use of Behavior Driven Development (BDD). We used SS-BDD to test three different Spreadsheet Programs. Our experience shows that SS-BDD can be used to build end-user friendly test scenarios which can achieve high fault-detection effectiveness.

CCS Concepts

• **Software and its engineering** → **Software organization and properties** → **Software creation and management** → **Software verification and validation** → **Software defect analysis**

Keywords

Spreadsheet Testing; Behavior Driven Development; Software Testing Tool.

1. INTRODUCTION

Current Spreadsheet Applications, such as Excel and Google Sheets, provide innumerable built-in facilities, including arithmetic, financial and statistical operations, as well as conditional expressions. Thus, end-users of Spreadsheet Applications with little or no formal training in programming can build their own “Spreadsheet Programs”, i.e., spreadsheets that calculate and display values based on complex domain-specific

models built using conditional expressions, formulas and data stored in tabular form (from hereafter “Spreadsheet Programs” are simply called “Spreadsheet”). In fact, control of software development is getting away from the hands of professionals and is passing to the hands of millions of domain experts [31]. And Spreadsheet Applications have become one of the most popular end-user programming environments nowadays.

Empowering end users to build their own computational solutions has several benefits [10], e.g., now, individuals and organizations use Spreadsheets for performing simple calculations or even to building complex financial models. However, it also leads to additional risks, including that Spreadsheets – as any other software – are prone to errors that are often the cause of considerable negative impacts on society [1][11][28][32]. Numerous works have concluded that Spreadsheets have a high number of faults. In [11], the authors confirm the frequency and seriousness of Spreadsheet errors. They observed at least one fault in every single Spreadsheet that was analyzed.

Unfortunately, commercial Spreadsheet Applications (e.g., Excel and Google Sheets) still provide limited support for helping end-user programmers (or. Spreadsheet programmers) to find faults. For example, Excel only provide the ability to calculate nested formulas one per time, to trace relationships between formulas and cells, and to watch a formula and its results. In contrast, researches have developed and proposed a number of sophisticated approaches [30] to prevent [2][15][16][25] and detect errors in Spreadsheets [4][12][13][17][18][20] [26]. For example, the Spreadsheet programmers can be educated to conduct systematic reviews and audits [2][4], or they can be even provided with additional software tools during the spreadsheet creation process [15][22][25]. Tools can help them detect errors and understand the reasons for the errors [15][22], and at least support the creation of spreadsheets free of errors by construction [19][25].

Spreadsheet supporting tools have specific design requirements that differ from those that guide the construction of typical software development tools. Spreadsheet programmers typically follow an unstructured development process, since they do not have formal training in programming. Indeed, they do not see themselves as doing programming. Instead, they build models by writing formulas that interrelate data disposed in tabular form. Tools should therefore be designed to support an incremental, unstructured and agile development process. Moreover, as Spreadsheets programmers are mostly non-professional programmers, testing approach in this context present a serious motivational challenge, because they require substantial effort on part of the end-user. This aspect is particularly relevant in the case of Spreadsheet construction since many of programmers are end-

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.

SAST’16, September 19–20, 2016, Maringá, Parana, Brazil.

© 2016 ACM. ISBN 978-1-4503-4766-2/16/09...\$15.00

DOI: <http://dx.doi.org/10.1145/2993288.2993296>

users who mainly want to get their job done. They are much less motivated than professional software developers to spend additional time on testing activities. Spreadsheet supporting tools should make it easy for a non-programmer to understand the value of investing the additional efforts. Therefore, testing approaches should not require any special training and should avoid special or low level terminology.

We approach the testing of Spreadsheets by adapting Behavior Driven Development (BDD) for the specification and execution of tests in the development of Spreadsheets. Behavior Driven Development was originally created by Dan North [8], and since then, has become an increasingly prevailing testing approach in practice [7][9]. The main advantage of applying BDD in the context of testing Spreadsheets is that it enables non-programmers write their own test cases by providing a high level textual descriptions of the requirements using specific keywords indicating the executable user's scenarios [8]. Scenarios describes how the software should behave when it is in a specific state and an event happens. The outcome of the scenario is an action that changes the state of the software or produces an output. Indeed, in general, scenarios are written by domain experts typically at the beginning of the software development activity and are considered as an executable form of the requirements, executed later by the developers. It also supports an adaptive cycle with fits well with the ad-hoc process often carried out by Spreadsheets programmers.

To support Spreadsheet programmers to apply test concepts to their daily tasks, we provide a BDD-based methodology that does not require an understanding of testing theory. This is accomplished through a test framework called SS-BDD (SpreadSheet Behavior Driven Development) which is a Gherkin-like testing framework¹ implemented in Java. It has been developed on top of JBehave² BDD framework and Apache POI³, a programming interface for manipulating Office Open XML files.

The main contributions of this paper are:

- We propose an approach to test Spreadsheets which addresses the user-friendly properties, requiring any special training and avoiding special or low level terminology
- We present a tool to support the implementation and automatic execution of tests developed according to our approach. Although the framework implementation is based on Excel, its underlying concepts can be applied to test Spreadsheets build in any application
- We use SS-BDD to test three Spreadsheets from different sources, and assess the quality of the test scenarios developed to these Spreadsheets using a fault injection technique. Our preliminary results show that SS-BDD provides effective tool support to build user-friendly test scenarios, which can achieve high fault-detection effectiveness.

The remainder of this paper is organized as follows. Section 2 presents a short background related to the current status of Spreadsheet testing approaches and Behavior Driven Development. Section 3 presents the testing approach and how the proposed concepts can be used to concretize some of its supporting ideas in the SS-BDD framework. Section 4 summarizes our experience of testing three Spreadsheets using SS-BDD and injecting faults to assess the test scenarios developed for these Spreadsheets. Section 5 provides further discussion of lessons learned. Finally, Section 6 presents our conclusions and directions for future work.

2. BACKGROUND

In this Section we present a short background related to the current status of Spreadsheet testing approaches and Behavior Driven Development.

	Product	Quantity	Unity Price	Total Price
3	Sweaters	3	\$ 18.99	\$ 56.97
4	Pants	8	\$ 22.99	\$ 183.92
5	Jacket	10	\$ 95.50	\$ 955.00
12	Total			\$ 1,195.89

Figure 1. The Stock Sheet - Inventory Control Spreadsheet in MS Excel.

¹ github.com/cucumber/cucumber/wiki/Gherkin

² jbehave.org

³ poi.apache.org

2.1 Spreadsheets

Spreadsheet can be found everywhere and are applied to solve complex problems for many professionals. Indeed, Spreadsheet Applications are probably the most used end-user programming environment on the planet [31]. We consider end-user programmers (or. Spreadsheet programmers) people who have little or no training at programming [10]. In contrast to general purpose programming languages, it is possible to create very useful Spreadsheets without training at programming [31]. Indeed, it is the software that has given everyone the means to play with numbers.

Yet based on a simple computation model, Spreadsheets Applications are powerful programming environments. They are mainly composed of cells, which can contain values (numbers, texts, dates, currency), references to other cells, or formulas. Cells are often disposed in a rectangular grid, composed of a quasi-infinity number of rows and columns. To refer to the contents of a cell, an unambiguous address has to be used. The most common form of cells address is the A1-style. Figure 1 illustrates a simple A1-style Spreadsheet. In this format, cells are referenced by its row and column position inside the Spreadsheet grid. For example, a cell at column B and row 3 would have the address B3. Groups of cells (i.e., cell ranges), can be also specified and used for example as formula input. The SUM(E3:E11) at cell E12 (Figure 1), sums the values in cells range (group) from E3 to E11.

From a design perspective, the content in Spreadsheets is usually arranged in tables. In this case, cells might contain simple labels. In the example (Figure 1), cells B2, C2, D2, E2, are *Label Cells*. They are used to name the column content. *Label Cells* can also be used as a form of documentation or to provide additional information about the overall Spreadsheet. Cells that contains any specific values (e.g., Numbers, Texts, Dates) and that are referenced by other cells are considered as *Input Cells*. For example, the Integer value “3” in C3 and the Currency value “\$ 18.99” in D3 are both inputs to the formula in E3 (=C3*D3). Finally, the cells that contains formulas constitute the Spreadsheet computation, and are trivially called *Formula Cells*. The cell F3 is a *Formula Cell* that computes the multiplication of the values placed in the *Input Cells* C3 and D3. These kind of cells are the ones which hold all Spreadsheet outputs. Considering the previous example, the *Formula Cell* E3 outputs the Total spent with a specific item in the stock.

In summary, Spreadsheets represents the logic, inputs and outputs, all at the same place. In this case, end-user programmers are able to create their programs by performing changes in any Spreadsheet cells (i.e., *Label Cells*, *Input Cells* and *Formula Cells*). However, each kind of change have distinctive impacts on the Spreadsheet. A change in a *Label Cells*, usually modifies the means of the value stored in near cells, while changes in *Formula Cells* produces modifications in the Spreadsheet computation. Changes in *Input Cells* are often only the ones allowed to Spreadsheet users, given that such a changes correspond to the way users interact with the Spreadsheet.

No matter the kind of change, Spreadsheets are in general used improperly or incorrectly, or without sufficient control [3][11][28][31]. Consequently, it has been show that Spreadsheets contain many errors [1][11][31], and that these errors are in many cases hard to discovery [32]. Although the exact definition of error varied among studies [29], in most cases, the emphasis is on formulas that produce incorrect values [4][22][29]. A possible factor in this problem is that Spreadsheet programmers usually are not aware of how to go about effectively find errors on their

Spreadsheets [23][27][29], and unfortunately, commercial spreadsheet tools (Excel and Google Sheets) do not provide any systematic mechanism for conducting tests.

2.2 Testing Spreadsheets

We consider testing as a controlled execution of a Spreadsheet, examining when it meets the programmer requirements. Typically, this means using known inputs and asserting the results against what would be expected from these inputs. In this case, a number of pioneering works in avoiding [16][17][19][18][25], finding [4][12][13][20][33] and fixing [24][15] Spreadsheets errors have been done by different research groups.

The “What You See is What You Test” methodology [33] conduct the testing of Spreadsheets by enabling the programmer to pronouncing output values as validated at any point during Spreadsheet development. If the output of a *Formula Cell* is correct, the programmer can mark it as correct. On the other hand, if the computed output is incorrect, the programmer can mark the cell as incorrect. An engine automatically tracks execution traces, which provide the information necessary to determine the cells which have influenced the output values (correct and incorrect). The main idea is to determine the cells that should be considered exercised as a result of this validation and to immediately communicate to the Spreadsheet programmer how well exercised the Spreadsheet are. So the programmer becomes aware of the level of correctness of their Spreadsheet through the cell border color on a continuum from red (untested) to blue (100% of the referenced cells have been exercised). With each validation that exercises a previously unexercised cell, the border becomes less red, indicating a greater degree of correctness for that cell. This visual feedback appears in all cells that contributed to the computation of the values in the validated cell.

In [12], the authors discuss the application of Test-Driven Development (TDD) [6] in the context of Spreadsheets. Basically, they proposed the application of TDD cycle on the development of Spreadsheets. Once the Spreadsheet programmers start creating their Spreadsheet, for example, creating a *Formula Cell*, they should first create a set of test cases which will exercise this unity of functionality. It is expected that, in the first moment, the recently created test case have to fail. Now, they construct the formula that should pass in the recently created test case. They, therefore, can continuously improve the formula and running the test case to exercise the Spreadsheet. The authors proposed a tool which support the automatic execution of test cases. The tool supports the automatic generation of simple test scripts, which allows the Spreadsheet programmers to focus on defining the valid test inputs and expected outputs.

Feature: Acquired items go to inventory
In order to keep track of the inventory values
As a store owner
I want to update the inventory value when new items are acquired
form suppliers

Scenario: The inventory value is updated

Given I have acquired new Sweaters
When 3 new are acquired by \$18.99 each one
Then the inventory value should be \$56.97

Figure 2. Gherkin-based Feature and Scenario Specification.

C - The Spreadsheet



Figure 3. The SS-BDD Process.

The presented efforts aim at some particular problems [13][20], propose one particular technique [12] and focus on one single optimization criterion such as maximizing the test coverage [33]. However, for testing of Spreadsheet to be applied in practice, it is important to include a discussion about the choice of comprehensible terminology and metaphors that can be used by end-user to express their teste cases scenarios easily and effectively. The open issues for Spreadsheet testing (i.e., how to empower the Spreadsheet programmers with user-friendly test scenarios that can achieve high fault-detection effectiveness?), which reflects in the lack of integrated user-friendly methodology and supporting tools, compose the motivation behind our work.

Behavior Driven Development, introduced by Dan North [8], is an engineering method to support delivery of software by improving communication between stakeholders, developers, and testers. By enhancing the communication among these teams, all involved people have a shared understating of software requirements and therefore software development meets user need more frequently.

This format is considered the natural way people in general (stakeholders, testers, and developers) talk about software requirements. Many authors [8][10] claim that, when clients are asked to exemplify how they expect a particular functional requirement, they naturally describe it in a sentence such as: “when I do that, then this should happen”. The Gherkin language, therefore, serves two purposes: serving as project’s documentation and automated tests. This is a good language to empower programmers with better abstraction mechanisms. This language should force end-user programmers to systematically reasoning about their problem firsts, avoiding faults and increasing the general quality of Spreadsheets. So, in next Section we present our testing approach, which supports the application of BDD concepts and ideas on the context of Spreadsheets.

This section presents an overview of the supporting ideas of our BDD-based Spreadsheet testing approach and discusses how some

of these ideas were concretized in the SS-BDD Framework. We use the Inventory Control SS to briefly demonstrate how SS-BDD can be used to test a Spreadsheet. The Inventory Control SS is a simple Spreadsheet comprised of three Sheets: Inventory, Sales, and Profit. The Inventory Sheet is a view of the current inventory level. It provides access to the quantities, unity price and inventory value of various products currently in stock. The Sales Sheet keeps track of all sales, recording the sold amount for each product. Finally, the Profit Sheet provides a view of the company profit, which includes, for each product in stock, the evaluation of the sold amount plus the unity price, considering a gain of 30%.

3.1 The SS-BDD Approach

As previously mentioned, our proposed testing approach defines test scenarios (i.e., inputs and expected values) using Ghecking as a specification language for end-user programmers. Therefore, the SS-BDD process begins with the user stories to be automated (see Figure 3 – A). A story is composed of several scenarios, each of which is composed of several steps. A step is an abstraction that represents contexts, events and actions. Their meaning is: in a particular case of a user story or context *C*, when event *E* occurs, the action of the system should be *A*. In Section 3.2 we describe the Scenarios Template that should be followed by the programmers when creating test scenarios.

In the next step, a test script is produced following the BDD pattern (see Figure 3 – B). Each **Given** has a procedure in charge of initializing the context. For example, defining which range of cells will be filled with the input values. Each **When** has a procedure to complete *Input Cells* with the input values. Finally, each **Then** has a procedure in charge of evaluating the *Formula Cells* and asserting the expected value against the obtained result. The test script can be manually created, but considering that end-user programmers do not have expertise on creating testing scripts on general purpose programming languages (e.g., Java), a more appealing solution is to automatically generate test scripts from test scenarios specified in the Ghecking language. In this case, the programmer explicitly defines in the scenario which cell should be completed with the input values. The end-user uses Spreadsheet cells names in the scenarios specification. In Section 3.2.1 we discuss the use of names in Spreadsheets and Parameters in scenarios specification. Section 3.4.1 illustrates a Test Script implemented on base in the SS-BDD framework.

Finally, the end-user programmers proceed with the Spreadsheet development (Figure 3 – C). As soon as they finish creating a specific Feature (e.g., Update the inventory with new items), they use the already created test scenarios to exercise the formulas and examine the Spreadsheet for any introduced fault (Figure 3 – D). The SS-BDD framework is used in this moment to automatically execute the test script and generate a fault report (Figure 3 – E). The report contains the scenarios that passed and the ones that failed. For each scenario, the used input values and the expected and observed outputs are exhibited.

In summary, the key elements of our Spreadsheet testing process are (see Figure 3):

1. The Scenarios which defines the conditions to which the Spreadsheet will be exposed and the expected values in such a conditions.
2. The Spreadsheet Under Test (SUT) which is the Spreadsheet whose behavior is to be verified by the execution of the Scenarios.

3. The Test Scripts which interact with the Spreadsheet Under Test completing the *Input Cells* with the input values specified in the scenarios and verifying whether the output values correspond to the expected ones also specified in the Scenarios.
4. The Execution Report which focus on reporting what tests have been executed, and whether they passed or failed. Moreover, it also includes the concept of “pending” tests. A pending test is one that has been specified, but which has not been implemented yet.

3.2 Scenarios Templates

The Gherkin Language format allows stockholders, developers and testers to use the same language to describe software’s behavior. However, when applied in the context of Spreadsheets, it might be hard for end-user programmers to understand how to expresses their intentions in this format. In order to provide some guidance in how the Gherkin Language works in the context of Spreadsheets, we proposed a Scenario Specification Template. One thing to be careful of when using the proposed template is that it could be automatically converted to Test Scripts (see Section 5.1). If this is the case, following the guidelines provided next is important, otherwise it will break the test execution.

Our recommendation is that each of scenarios for the story must have a title. The title should briefly explain what activity the users will be able to complete when they finish the scenario. Following this guideline, the end-user programmer will have an idea of what the finished Scenario should look like. Figure 4 provides an example Title for the “Acquired items must be added to stock” scenario elaborated to exercise the Inventory Control SS.

Scenario: Acquired items must be adding to stock

Given I have acquired new <Stock!Product>
When <Stock!Quantity> new are acquired by
 <Stock!UnityPrice> each one
Then the inventory value should be <Stock!Total>

Examples:

<Stock!Product>	<Stock!Quantity>	<Stock!UnityPrice>	<Stock!Total>
Sweaters	3	18.99	56.97
Pants	8	22.99	183.92
Jacket	10	95.50	955.00

Figure 4. Scenario Specification in JBehave.

First, every Scenario has to simulate several users input via Scenarios Outlines. This decision allows Spreadsheet programmers to concisely express several examples through the use of a Template with Parameters. In this case, the Scenarios are written as a template that is never directly run. A Scenario Outline is run once for each row in the Examples Table. For example, the Scenario “Acquired items must be adding to stock”, illustrated in Figure 4, will be executed for each line specified in the Examples Table (Sweaters, Pants, Jacket). Moreover, as can be seen in Figure 4, the Scenarios introduces Parameters (e.g., <Stock!Product>, <Stock!Quantity>), which are contained within “< >” in the Scenarios Step specification. We can consider Parameters as a variable that is replaced with a real value from the Example Table during the test execution.

In the context of testing of Spreadsheets, the purpose of **Given** steps is to put the SS-BDD in a known state before the user

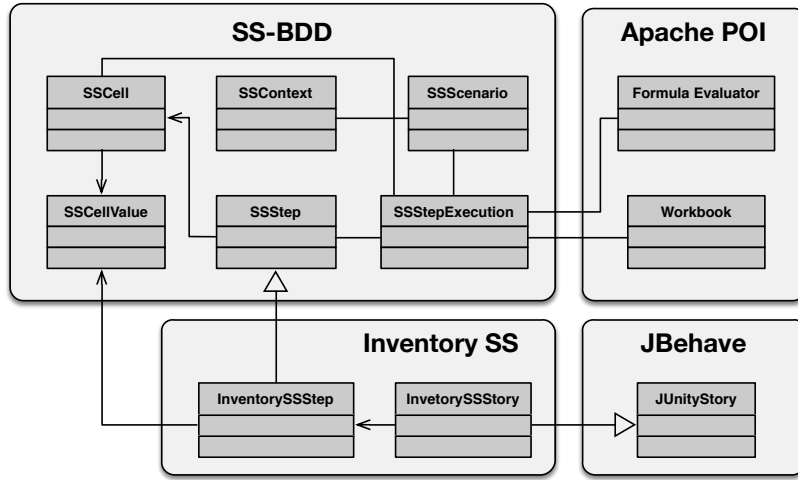


Figure 5. The SS-BDD Framework Design.

starts interacting with the Spreadsheet. For example, in Figure 4, the **Given** step informs the SS-BDD to complete the *Input Cells* associated to the Sweaters product in the Stock Sheet. Therefore, it is important to avoid talking about user interaction in **Given** steps. The purpose of **When** steps is to describe all associated inputs the user performs on the Spreadsheet and the purpose of **Then** steps is to evaluate *Formula Cells* and assert the outcomes. In the example on Figure 4, the **When** step indicates that the *Input Cells* “Stock!Quantity” and “Stock!UnityPrice” have to be completed with the values “3” and “\$18.99”, respectively; and the **Then** step indicates that the “Stock!Total” *Formulas Cell* should be evaluated and that the expected values are “\$56.97”. The outcomes in **Then** steps should be related to the business value in Feature descriptions. The Scenarios should focus on inspecting the output of *Formula Cells* and not something different (validate inputs or checking for design decisions).

3.2.1 Names in Spreadsheets and Parameters in Scenarios Template

In Spreadsheets design, a name is an evocative shorthand that makes it easier for users to understand the purpose of a cell (or a range of cells), constant, formula, or table, each of which may be difficult to comprehend at a first look. Therefore, by using names, Spreadsheet programmers can make their formulas much easier to understand, update, audit and manage. The Figure 1 shows how names can be created on Spreadsheets and how they can improve clarity and understanding.

In Spreadsheets, all names have a scope. The scope of a name is the location within which the name is recognized without qualification. For example, in Figure 1, the scope of the name Product is Stock Sheet. That name, if not qualified, is recognized only in Stock Sheet, but not in other sheets without qualification. To use a local worksheet name in another worksheet, you can qualify it by preceding it with the worksheet name, as the following example shows: “Stock!Product”. On the other hand, if we have defined a name, such as Product, and its scope is the workbook, that name is recognized for all worksheets in that workbook, but not for any other workbook.

A name must always be unique within its scope, therefore, Spreadsheets applications in general prevents programmers from defining a name that is not unique within its scope. However, they

can use the same name in different scopes. For example, they can define a name, such as “Product” that is scoped to “Stock”, “Sales”, and “Profit” in the same workbook. Although each name is the same, each name is unique within its scope. End-user programmers might do this to ensure that a formula that uses the name, “Product”, is always referencing the same cells at the local worksheet level.

Finally, programmers can even define the same name, “Product”, for the global workbook level, but again the scope is unique. In this case, however, there can be a name conflict. To resolve this conflict, by default some Spreadsheet applications (e.g. Excel) uses the name that is defined for the worksheet because the local worksheet level takes precedence over the global workbook level. When the programmers want to override the precedence to use the workbook name, they can disambiguate the name by prefixing the workbook name (e.g., “WorkbookFile!Product”).

Based on the observation that Spreadsheet users are motivated to place names for ranges into Spreadsheets as a meaningful abstraction, we can convention the combination of these named cells and parameters in scenarios templates. It decreases the number of decisions that a Spreadsheet programmer using the SS-BDD is required to make without necessarily losing flexibility. For example, if there is a named cells Product in the Spreadsheet Stock, the corresponding parameter in the scenario template should be “Stock!Product” by default. As a consequence, as presented in Section 3.4.1, a lot of (tedious and repetitive) explicit code is replaced by simple code accessing cells range by the name convention.

3.3 Fault Model

Testing approaches should focus on a specific fault model [34]. The fault model defines a subset of types of faults considered by the testing approach, which comprises its fault detection capabilities and hence delimits the kinds of faults it aims at detecting [34]. In Spreadsheets, errors might have two distinct causes [5][11][29]: (i) system-generated errors; or user-generated errors. System-generated errors are beyond the control of users, although, they can, when aware, take corrective actions. User-generated errors, in contrast, are introduced by the user and can be prevented, detected and corrected by them.

Rajalingham et al. [5] and Powell et al. [29] classify user-generated errors as quantitative or qualitative errors. Qualitative errors do not immediately produce incorrect values but degrade the quality of the Spreadsheet. Quantitative errors, on the other hand, are errors that lead to incorrect values. They are mainly introduced by end-user programmers and are generated by omissions, duplications, incomplete mathematical representations, logic errors and so on. Based on such a classification, we have defined an initial candidate fault model for our testing approach, composed a set of faults caused by Spreadsheet programmers which can be manifested as a failure on the execution of the formulas and consequently impair their calculations, are they: (i) incorrectly constructed formulas; (ii) relative and absolute copy problems; (iii) misconception of built-in functions.

3.4 Framework Design and Implementation

Figure 5 depicts the main elements of SS-BDD. The `SSStep` implements the Step concept in SS-BDD. Instead of creating a BDD-based testing tool from scratch to build and execute Spreadsheets testing scenarios, we extend JBehave, a framework for BDD implemented in Java. Every SS-BDD Test Script has to extend the `SSStep` class. This class offers a set of helper methods to ease the implementation of test scripts. These methods are:

- `cell([RangeName])` - provides access to a cell by its name and returns a `SSCell` instance.
- `context([RangeName],[Value])` - initializes the test execution context (available to **When** and **Then** steps), with the range of cells or line in a Spreadsheet table that should be completed with the input values.

The `SSCell` class abstracts the access to concrete Spreadsheet cells. It can be used to complete *Input Cells* with values or to evaluate a *Formula Cells* and obtain the resulting value. The `SSCell` class interfaces with the Apache POI API to open, read, and write values in the Spreadsheet Under Test. This class also offers a set of helper methods that ease the location of cells by name, or even by a reference cell. Once you have a cell at hands it is possible to change its value (method `value()`), or it is also possible, in case of *Formula Cells*, to evaluate it and observe the output, (method `evaluate()`). This method returns a `SSCellValue` which abstract the access to the output value. The output value might be of the following types: Text, Number, Currency and Date.

```

01 public class SSCSSStep extends SSStep {
02
03     @Given("I have acquired new <Stock!Product>")
04     public void gv(@Named("Stock!Product")String sp){
05         context("Stock!Product", sp);
06     }
07
08     @When("<Stock!Quantity> new are acquired by
09           <Stock!UnityPrice> each one ")
10     public void wh(@Named("Stock!Quantity") int sq,
11                   @Named("Stock!UnityPrice") double sp){
12         cell("Stock!Quantity").value(sq);
13         cell("Stock!UnityPrice").value(sp);
14     }
15
16
17     @Then("the inventory value should be <Stock!Total>")
18     public void th(@Named("Stock!Total") double st){
19         SSCellValue stCellValue = cell("Stock!Total").evaluate()
20         Assert.assertEquals(precoTotal,stCellValue.getValue(),0);
21     }

```

Figure 6. A `ssstep` Test Script Code.

3.4.1 Test Cases Implementation

Spreadsheet test scripts consist of a set of scenarios (Gherkin scenarios) implemented as candidate steps, i.e., Java methods annotated with `@Given`, `@When` and `@Then` tags in a class that extends `SSStep` (see Figure 6). Basically, `@Given` methods initialize the scenario execution context, `@When` methods interact with Spreadsheet Under Test by storing values in the *Input Cells*; and `@Then` methods are in charge of evaluate *Formula Cells* and assert the observed values with the expected ones.

Figure 6 illustrates a test script for the Scenario depicted in Figure 4. According to the code in Figure 6, firstly, the test case is prepared (Line 5) by initializing the execution context with the value placed in the `Stock!Product` column in the Example Table. This value is communicated by means of the `sp` parameter in the `@Given` method. Next, the *Input Cells* are completed through methods available in the SS-BDD Framework (Lines 13-14). Finally, the `@Then` method evaluates the *Formula Cell* (Line 19) and checks the results against the expected one (Line 20). The `SSCellValue` carries the evaluated value, which is asserted with the expected value originated from the Example Table (Figure 6).

3.4.2 Teste Script Execution Control

As said before, Scenarios Outlines allows end-user programmers to more concisely express several examples that run once for each row in the Examples Table. In order to effectively test a Spreadsheet – given that tests are based on Scenarios Outlines – we need to control exercise of the Spreadsheet Under Test. To keep track of a scenarios execution state transitions the SS-BDD Framework introduces a SS Execution Control component (`SSStepExecution`). This component maintains the current state of the steps execution in a set of internal data structures, and makes this information available to other frameworks components (e.g.: `SSCell`). This component helps the `SSCell` to know, for example, which specific cell in a named range should be completed with the values from the Example Table.

For example, consider the execution of the test script in Figure 6. The **Given** in Figure 5 refers to the “Stock!Product”, and, according to the Example Tables, its initial context execution value is “Sweaters” (Line 5 – Figure 6). Therefore, in the execution of the **When**, the Execution Control component knows the Spreadsheet line related with the context value (Line 3 in the Spreadsheet, see Figure 1) and guides the `SSCell` to return the cell in the range “Stock!Quantity” associated with “Sweaters” quantity. In this example, the C3 cell. Finally, in the **Then**, the `SSCell` also returns the cell in the range “Stock!Total” associated with the “Sweaters” Total Price (i.e., E3). Following the same rationality, in next execution, the **Given** in Figure 5 now refers to “Paints” as context execution value (second line in the Example Table). The Execution Control therefore guides the `SSCell` to return cells in the ranges “Quantity”, “UnityPrice” and “Total” associated with the “Paints” product (i.e., cells in Line 4 in the Spreadsheet, see Figure 1).

4. EVALUATION

This Section presents our experience in testing Spreadsheets using the SS-BDD Framework. We present three Spreadsheets from different domains that were tested using SS-BDD. Due to space limitation, we presented how a test scenario can be built and executed in the context of one of the Spreadsheet – Inventory Control SS (Section 3); the other test scenarios essentially follow the same structure. Finally, we present the results obtained from

testing the three Spreadsheets and our evaluation of SS-BDD testing capabilities.

4.1 Spreadsheets

Our Spreadsheets includes: a Simple Inventory Control SS (IC); a Homebrew Control SS (HC), and a Self-service Control SS (SC). The Homebrew Control Spreadsheet helps brewers control the production of their own beer. They can control the amounts of grains and malt used in each recipe, estimates the costs of each produced beer, and control the level of the current stock. The Self-service Control SS is a traditional Spreadsheet applied to control the production and profits in self-service restaurants.

Table 1. Faults detected when testing Inventory Control (IC), Homebrew Control (HC) and Self-service Control (SC) Spreadsheets

Fault Type	Detected Faults			
	IC	HC	SC	All
Incorrectly Constructed Formulas	1	5	4	10
Relative and Absolute Copy Problems	-	8	9	17
Misconception of Build-in Functions	-	4	7	11
Total	1	17	20	38

Table 2. Faults injected and detected

Fault Type	Injected/Detected Faults			
	IC	HC	SC	All
Incorrectly Constructed Formulas	4/4	4/4	4/4	12/12
Relative and Absolute Copy Problems	4/3	4/2	4/2	12/7
Misconception of Built-in Functions	4/4	4/4	4/4	12/12
Total	12/11	12/10	12/10	36/31

4.2 SS-BDD in Practice

We progressively tested the three Spreadsheets using the SS-BDD Framework. First, we defined Scenarios to each case of User Input, based on the guidelines presented in Section 3.2, and on the error-guessing test case design technique [35]. Figure 3 illustrated an example of which we consider as User Input (Acquired items go to inventory). Second, we manually implemented some Test Scripts, one for each Scenario. Figure 6 presents an example of Test Script. Finally, we execute all the implemented Test Scripts. As a result, we found 38 faults during the execution of these Scripts. These faults are summarized on Table 1.

As one can observe in Table 1, from a total of 38 faults, 10 were related to Incorrectly Constructed Formulas (e.g., the Current Inventory Total was evaluating the wrong Item) and 17 were related to Relative and Absolute Copy Problems on Self-service Control SS. The other 11 faults were related to Misconception of Built-in Functions. Some formulas did not agree on the right use of the IF Function. For example, there were cases that were referring to Empty Cells. Another interesting finding during our tests was that most of the *Formula Cells* were not created to deal with exceptional conditions (e.g., division by zero). A reason may reside on the fact that end-user programmers almost always have in mind the main conditions and neglect “what should the Spreadsheet do if a user provides a wrong input?”. Thus, it is up to the Spreadsheet user to decided what should be provided. As a consequence, in most of cases the user does not follow the expected input conditions. The error-prone test selection

technique in combination with the guidelines presented in Section 3.2 was a valuable tool to examine how the Spreadsheets behave under these exceptional scenarios, and was responsible for uncovering most of the faults.

4.3 Evaluation

We have randomly injected some faults [36] in all three Spreadsheets in order to try to estimate the effectiveness of the test cases developed using the SS-BDD approach. We proceed by injecting faults specified in our fault model (see Section 3.3). We have injected 36 faults inside formulas spread over all Worksheets to check whether the tests scenarios were able to diagnose the injected faults. Table 2 summarizes the results of our fault injection experiment.

Every incorrectly constructed formula was precisely detected by one of the created test scenarios. This percentage assures the quality of the test scenarios implemented. The reason for such results can be twofold. Firstly, because the test scenarios have been developed carefully, and continuous improved and maintained. Secondly, because the Spreadsheet programmers have been acquiring expertise in developing the test scenarios and, as a consequence, each new scenario have become more effective. Some relative and absolute copy problems were not detected by the test scenarios (5 faults); some of them (4 faults) because they were not exercised by the Scenario, the number of elements in the Example Table was not enough to exercise the injected problems, other (1 fault) because there was no test scenario that exercised some absolute copy problems. Concerning the misconception of build-in functions, every time the use of a function was corrupted it was detected by the test scenario. This can be explained due to the possibility of different Test Scenarios for exercising one specific combination of functions.

5. DISCUSSIONS

This section provides further discussion of issues and lessons we have learned while using SS-BDD to test the previously described Spreadsheets.

5.1 Automatic Generation of Test Scripts

Considering that end-user programmers do not have any expertise on programming, the automatic generation of test scripts is a mandatory requirement. In this case, instead of obligate the implementation of test scripts, a more appealing solution is to permit the end-user programmers exercise their Spreadsheets directly from scenarios specification. The guidelines presented in Section 3.1 supports the development of a template-based technique to automatically generate Test Scripts from scenarios.

The generative technique should first trace all steps in scenarios and generate corresponding Java methods, one for each **Given**, **When** and **Then**. Considering that each **Given** step initialize the scenarios context by defining which specific part of a range of cell should be completed with the input values, the generator have to include in every @Given method a call to `context([RangeName],[Value])`, passing, respectively: (i) the Step Parameter name; and (ii) the value provided in the @Named method parameter (see Line 5 in Figure 6). Next, for each @When method, and in this case, for each Step Parameter, the generator has to include a call to `cell([RangeName]).value([Value])`, passing, respectively: (i) the Step Parameter Name; (ii) and the corresponding value provided in the @Named method parameter (see Lines 13 and 14 in Figure 6). Finally, for each @Then method, a call to `cell([RangeName]).evaluate()`, have to be included, receiving as well as, the Parameter Name. And, it also

has to include `assertEquals([ExpectedValue], [ObservedValue])`, statement responsible for comparing the observed value with the one provided in the `@Named` method parameter (expected value) (see Lines 19 and 20 in Figure 6).

5.2 Test Adequacy Criterion for Spreadsheets

Sometimes the Spreadsheet behavior may not be sufficiently exercised by the scenarios initially created by the end-user programmer. In general, they do not understand the technicalities of the test methodologies or the technical implications of the faults they are dealing with. Moreover, they have no imagination in how to build good test scenarios and most of their efforts can be concentrated on how to make things work right and effectively. The mental switch required to move them from a positive mindset to a what-can-go-wrong mindset is not trivial and very hard to achieve for end-user programmers [32]. Therefore, works such as [14][21][22][37] have used coverage results not only to give feedback to programmers on what parts of code are to be exercised, but also give further guidance on how to improve the test coverage. The use of coverage information can be adapted to the context of Spreadsheet development and become valuable as a feedback to improve effectiveness of the test scenarios defined using SS-BDD.

5.3 Excel and Google Sheets Add-Ins

Spreadsheet applications (e.g., Excel and Google Sheets) are characterized by incremental visual feedback that is tangled with the construction process. The most interesting example of this is the automatic recalculation functionality. This incremental visual feedback motivates the application of testing methodologies that support an incremental construction of the test scenarios and an on-the-fly execution process. For example, when a user changes a formula, the testing framework should provide feedback about how this affects the correctness of each affected portion of the Spreadsheet. This raises the confidence of dealing with evolving Spreadsheets. As the SS-BDD is based on Java and developed over JBehave, it does not support on-the-fly execution and feedback for end-user programmers. However, its underling concepts (Section 3) can be applied to test any Spreadsheet, given that the Spreadsheet application supports Add-Ins, has an existing BDD Framework for the implementation language and API available to read and modify cells and evaluate formulas. Excel and Google Sheets are two examples of Spreadsheets tools that can be extended by Add-ins, there is a BDD Framework for their Add-in implementation language, and provides API to read and modify cells and evaluate formulas.

Google Sheets Add-ins are developed in JavaScript. Indeed, there are a JavaScript implementation of Cucumber⁴, another BDD Framework, initially developed to automate acceptance testing for Ruby applications. This framework can be embedded as Google Scripts⁵ and used to implemented our proposed approach as Add-in for Google Sheets. SpecFlow⁶ is another BDD Framework specially developed to automate acceptance testing for .Net applications. Considering that Excel extensions can be developed in VB or C# (.Net compatible languages), our proposed approach can be easily migrated to interact with SpecFlow instead of JBehave, and directly used the Excel provided API, in place of interact with the Apache POI.

⁴ cucumber.io

⁵ www.google.com/script/start/

⁶ www.specflow.org

5.4 User-friendly Reports

Conventional reports, generated by tools such as JUnit⁷, focus on what tests have been executed, and whether they passed or failed. While this is certainly valuable from a testing perspective, these reports are far from telling the whole history. BDD tools such as JBehave not only report on test results, but they also report on the user stories that these tests validate, the inputs and the observed outputs. However, this reporting is still limited for Spreadsheet development, where the results can become cumbersome to be understood by end-user programmers. So it makes sense to be able to report on test at higher levels, for example in terms of Spreadsheet capabilities (e.g, formulas or computations). This makes it easier to report on not only what scenarios have been implemented and exercised, but also what capabilities (formulas) remain to be completed. Therefore, the SS-BDD can be integrated to acceptance testing frameworks such as Serenity⁸ in order to provide more comprehensive and user-friendly reports.

6. CONCLUSION AND FUTURE WORK

We have presented SS-BDD, a framework for building and running tests for Spreadsheets. The test scenarios in SS-BDD are based on the concepts of Behavior Driven Development to orient end-user programmers in how to effectively write good test cases. SS-BDD has been used to test three Spreadsheets from different sources. In these experiments, SS-BDD was used as a generic testing framework for Excel Spreadsheets.

End-user programmers now can benefit from a set of automatic tests that can run constantly, improving the confidence in the Spreadsheet and even supporting the maintainability tasks. Although SS-BDD automatic tests could not assure the absence of faults in the tested Spreadsheets, they could guarantee that whenever scenarios were executed, they had executed as expected. Moreover, we also used a fault injection approach to asses the quality of the test scenarios developed to each one of these Spreadsheets. Our preliminary results have shown that SS-BDD can effectively uncover bugs related to Spreadsheet fault model. In future work, we plan to implement a generative technique: generating test scripts; as well as leveraging existing test generation tools [22][37] to generate set of test inputs for Spreadsheet. Finally, we also plan to concretize our proposed approach as two Add-Ins, one for Excel and another to Google Sheets.

7. REFERENCES

- [1] S. Ditlea. Spreadsheets Can be Hazardous to Your Health. *Personal Computing*, 11(1):60–69, 1987.
- [2] M. M. Burnett, C. Cook, J. Summet, G. Rothermel, and C. Wallace. End-User Software Engineering with Assertions. *IEEE Int. Conf. on Software Engineering*, pp. 93–103, 2003.
- [3] R. R. Panko. Spreadsheet Errors: What We Know. What We Think We Can Do. *Symp. of the European Spreadsheet Risks Interest Group (EuSpRIG)*, 2000.
- [4] G. Rothermel, M. M. Burnett, L. Li, C. DuPuis, and A. Sheretov. A Methodology for Testing Spreadsheets. *ACM Transactions on Software Engineering and Methodology*, pp. 110–147, 2001.

⁷ junit.org

⁸ www.thucydidides.info

- [5] K. Rajalingham, D. R. Chadwick, and B. Knight. Classification of Spreadsheet Errors. *Symp. of the European Spread-sheet Risks Interest Group (EuSpRIG)*, 2001.
- [6] D. Janzen, D.H. Saiedian, test-driven development: concepts, taxonomy, and future directions, *Computer*, vol.38, no. 9, pp. 43-50, Sept, 2005.
- [7] D. Chelimsy, D. Astels, Z. Dennis, A. Hellesoy, D. North. The RSpec book: Behaviour Driven Development with RSpec, cucumber and friends, Pragmatic Bookshelf, 2010.
- [8] D. North, Introducing BDD, 2006. Available at: <http://dannorth.net/introducing-bdd> [Accessed July 07, 2016].
- [9] C. Solis and X. Wang, "A Study of the Characteristics of Behaviour Driven Development," *2011 37th EUROMICRO Conference on Software Engineering and Advanced Applications*, Oulu, 2011, pp. 383-387.
- [10] Burnett, M., Cook, C., Rothermel, G.: End User Software Engineering. Communications of the ACM 47(9) (September 2004)
- [11] Panko, R.: What We Know About Spreadsheet Errors. Journal of End User Computing 10(2), 15–21 (Spring 1998)
- [12] Alan Rust, Brian Bishop, Kevin McDaid. Test-Driven Development: Can It Work for Spreadsheet Engineering?. Extreme Programming and Agile Processes in Software Engineering. LNCS. Volume 4044. pp 209-210. 2006.
- [13] Chris Chambers and Martin Erwig. Automatic detection of dimension errors in spreadsheets. *J. Vis. Lang. Comput.* Volume 20, Issue 4, 269-283. 2009
- [14] M. Fisher, G. Rothermel, D. Brown, M. Cao, C. Cook, and B. Burnett. Integrating Automated Test Generation into the WYSIWYT Spreadsheet Testing Methodology. *ACM Trans. on Software Engineering and Methodology*, 2006.
- [15] R. Abraham and M. Erwig. Goal-Directed Debugging of Spreadsheets. In *IEEE Int. Symp. on Visual Languages and Human-Centric Computing*, pages 37–44, 2005.
- [16] R. Abraham, M. Erwig, S. Kollmansberger, and E. Seifert. Visual Specifications of Correct Spreadsheets. In *IEEE Int. Symp. on Visual Languages and Human-Centric Computing*, pages 189–196, 2005.
- [17] Y. Ahmad, T. Antoniu, S. Goldwater, and S. Krishnamurthi. A Type System for Statically Detecting Spreadsheet Errors. In *18th IEEE Int. Conf. on Automated Software Engineering*,
- [18] T. Antoniu, P. A. Steckler, S. Krishnamurthi, E. Neuwirth, and M. Felleisen. Validating the Unit Correctness of Spreadsheet Programs. In *26th IEEE Int. Conf. on Software Engineering*, pages 439–448, 2004.
- [19] G. Engels and M. Erwig. ClassSheets: Automatic Generation of Spreadsheet Applications from Object-Oriented Specifications. In *20th IEEE/ACM Int. Conf. on Automated Software Engineering*, pages 124–133, 2005.
- [20] M.M. Burnett, A. Sheretov, B. Ren, G. Rothermel, Testing homogeneous spreadsheet grids with the “What You See Is What You Test” methodology, *IEEE Transactions on Software Engineering* 29 (6) (2002) 576–594.
- [21] M. Fisher II, M. Cao, G. Rothermel, C. Cook, M.M. Burnett, Automated test case generation for spreadsheets, in: *IEEE International Conference on Software Engineering*, 2002, pp. 141–151.
- [22] R. Abraham, M. Erwig, AutoTest: a tool for automatic test case generation in spreadsheets, in: *IEEE International Symposium on Visual Languages and Human-Centric Computing*, 2006, pp. 43–50.
- [23] J. Lawrence, R. Abraham, M.M. Burnett, M. Erwig, Sharing reasoning about faults in spreadsheets: an empirical study, in: *IEEE International Symposium on Visual Languages and Human-Centric Computing*, 2006, pp. 35–42.
- [24] T. Antoniu, P.A. Steckler, S. Krishnamurthi, E. Neuwirth, M. Felleisen, Validating the unit correctness of spreadsheet programs, in: *26th IEEE International Conference on Software Engineering*, 2004, pp. 439–448.
- [25] R. Abraham, M. Erwig, UCheck: a spreadsheet unit checker for end users, *Journal of Visual Languages and Computing* 18 (1) (2007) 71–95.
- [26] Michael J. Coblenz, Andrew J. Ko, Brad A. Myers, Using objects of measurement to detect spreadsheet errors, in: *IEEE Symposium on Visual Languages and Human-Centric Computing*, 2005, pp. 314–316.
- [27] P. Brown, J. Gould, An experimental study of people creating spreadsheets, *ACM Transactions on Office Information Systems* 5 (1987) 258–272.
- [28] J. Caulkins, E. Morrison, T. Weidemann, Spreadsheet errors and decision making: evidence from field interviews, *Journal of End User Computing*. 2006
- [29] Stephen G. Powell, Kenneth R. Baker, and Barry Lawson. 2008. A critical review of the literature on spreadsheet errors. *Decis. Support Syst.* 46, 1 (December 2008), 128-138.
- [30] Dietmar Jannach, Thomas Schmitz, Birgit Hofer, Franz Wotawa, Avoiding, finding and fixing spreadsheet errors – A survey of automated approaches for spreadsheet QA, *Journal of Systems and Software*, Volume 94, August 2014, Pages 129-150, ISSN 0164-1212.
- [31] Panko, R.R., Port, D.N., 2012. End User Computing: The Dark Matter (and Dark Energy) of Corporate IT. In: *Proceedings of the 45th Hawaii International Conference on System Sciences (HICSS 2012)*, Wailea, HI, USA, pp. 4603–4612.
- [32] Galletta, D.F., Abraham, D., Louadi, M.E., Lekse, W., Pollalis, Y.A., Sampler, J.L., 1993. An empirical study of spreadsheet error-finding performance. *Account. Manage.*
- [33] Rothermel, G., Li, L., Dupuis, C., Burnett, M., 1998. What you see is what you test: a methodology for testing form-based visual programs. In: *Proceedings of the 20th International Conference on Software Engineering (ICSE 1998)*, Kyoto, Japan, pp. 198–207.
- [34] Binder, R., *Testing Object-Oriented Systems. Models, Patterns, and Tools*, Addison-Wesley, 1999.
- [35] Myers, G.J., *The Art of Software Testing*, Wiley, 2004.
- [36] Voas, J. and McGraw, G. *Software Fault Injection: Inoculating Programs Against Errors*. Wiley, 1998.
- [37] Xie, T., Zhao, J., A Framework and Tool Supports for Generating Test Inputs of AspectJ Programs. *AOSD’06*, 2006.